Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования «Санкт-Петербургский национальный исследовательский университет информационных технологий, механики и оптики»

Факультет «Программной Инженерии и Компьютерной Техники»

Дисциплина «Системное программное обеспечение»

Отчёт к практическому заданию № 3

Вариант 22

Входной язык 2

Выполнил:

Студент группы P4114 Тягельский А. А.

Проверил:

Доцент факультета ПИиКТ Кореньков Юрий Дмитриевич

Санкт-Петербург, 2025

Цель работы:

Реализовать формирование линейного кода в терминах некоторого набора инструкций посредством анализа графа потока управления для набора подпрограмм. Полученный линейный код вывести в мнемонической форме в выходной текстовый файл. Характеристики виртуальной машины согласно варианту: статическая типизация, стековый формат инструкций ВМ, 2 банка памяти под code, constants и data.

Задачи:

1. Составить описание виртуальной машины с набором инструкций и моделью памяти по варианту:
    a. Изучить нотацию для записи определений целевых архитектур.
    b. Составить описание ВМ в соответствии с вариантом.
        i. Описание набор регистров и банков памяти.
        ii. Описать набор инструкций: для каждой инструкции задать структуру операционного кода, содержащего описание операндов и набор операций, изменяющих состояние ВМ.
            1. Описать инструкции перемещения данных и загрузки констант.
            2. Описать инструкции арифметических и логических операций.
            3. Описать инструкции условной и безусловной передачи управления.
            4. Описать инструкции ввода-вывода с использованием скрытого регистра в качестве порта ввода-вывода.
        iii. Описать набор мнемоник, соответствующих инструкциям ВМ.
    c. Подготовить скрипт для запуска ассемблированного листинга с использованием описания ВМ:
        i. Написать тестовый листинг с использованием подготовленных мнемоник инструкций.
        ii. Задействовать транслятор листинга в бинарный модуль по описанию ВМ.

        iii. Запустить полученный бинарный модуль на исполнение и получить результат работы.
        iv. Убедиться в корректности функционирования всех инструкций ВМ.
2. Выбрать и изучить прикладную архитектуру системы команд существующей ВМ.
    a. Для выбранной ВМ:
        i. Должен существовать готовый эмулятор (например qemu).
        ii. Должен существовать готовый тулчейн (набор инструментов разработчика): компилятор Си, ассемблер и дизассемблер, линковщик, желательно отладчик.
    b. Согласовать выбор ВМ с преподавателем.
    c. Изучить модель памяти и набор инструкций ВМ.
    d. Научиться использовать тулчейн (собирать и запускать программы из листинга).
    e. Подготовить скрипт для запуска ассемблированного листинга с использованием эмулятора.
        i. Написать тестовый листинг с использованием инструкций ВМ.
        ii. Задействовать ассемблер и компоновщик из тулчейна.
        iii. Запустить бинарный модуль на исполнение и получить результат его работы.

Порядок выполнения:

1. Описать структуры данных, необходимые для представления информации об элементах образа программы (последовательностях инструкций и данных), расположенных в памяти.
    a. Для каждой инструкции – имя мнемоники и набор операндов в терминах данной ВМ.
    b. Для элемента данных – соответствующее литеральное значение или размер экземпляра типа данных в байтах.
2. Реализовать модуль, формирующий образ программы в линейном коде для данного набора подпрограмм.
    a. Программный интерфейс модуля принимает на вход структуру данных, содержащую графы потока управления и информацию о локальных переменных и сигнатурах для набора подпрограмм, разработанную в задании 2 (п. 1.a, п. 2.b).

b. В результате работы порождается структура данных, разработанная в п. 1, содержащая описание образа программы в памяти: набор именованных элементов данных и набор именованных фрагментов линейного кода, представляющих собой алгоритмы подпрограмм.
c. Для каждой подпрограммы посредством обхода узлов графа потока управления в порядке топологической сортировки (начиная с узла, являющегося первым базовым блоком алгоритма подпрограммы), сформировать набор именованных групп инструкций, включая пролог и эпилог подпрограммы (формирующие и разрушающие локальное состояние подпрограммы).
d. Для каждого базового блока в составе графа потока управления сформировать группу инструкций, соответствующих операциям в составе дерева операций.
e. Использовать имена групп инструкций для формирования инструкций перехода между блоками инструкций, соответствующих узлам графа потока управления, в соответствии с дугами в нём.

3. Доработать тестовую программу, разработанную в задании 2 для демонстрации работоспособности созданного модуля.
   a. Добавить поддержку аргумента командной строки для имени выходного файла, вывод информации о графах потока управления сделать опциональных.
   b. Использовать модуль, разработанный в п. 2 для формирования образа программы на основе информации, собранной в результате работы модуля, созданного в задании 2 (п. 2.b).
   c. Для сформированного образа программы в линейном коде вывести в выходной файл ассемблерный листинг, содержащий мнемоническое представление инструкций и данных, как они описаны в структурах данных (п. 1), построенных разработанным модулем (пп. 2.c-e).
   d. Проверить корректность решения посредством сборки сгенерированного листинга и запуска полученного бинарного модуля на эмуляторе ВМ (см. подготовка п. 1.c или п. 2.e)
4. Результаты тестирования представить в виде отчета, в который включить:

a. В части 3 привести описание разработанных структур данных.
b. В части 4 описать программный интерфейс и особенности реализации разработанного модуля.
c. В части 5 привести примеры исходных текстов, соответствующие ассемблерные листинги и примера вывода запущенных тестовых программ.

Ход работы

1 Описание структуры данных для представления информации об элементах образа программы

Для запуска ассемблированного листинга необходимо описать целевую архитектуру, для чего разработан target-definitions.pdsl, расположенный ниже.

```
architecture MyVM {
	registers:
		/* Instruction Pointer: points to CODE_CONST bank (byte address) */
		storage IP[32];
		view ip = IP;

		/* Stack Pointer: index (or byte addr) in DATA bank */
		storage SP[32];
		view sp = SP;

		/* Frame Pointer: base for locals */
		storage FP[32];
		view fp = FP;

		/* Hidden IO port register (IN/OUT use it) */
		storage IOPORT[16];
		view ioport = IOPORT;

		/* Hidden IO data registers */
		storage STDIN[32];
		storage STDOUT[32];
```

```
        view stdin = STDIN;
        view stdout = STDOUT;

        /* One-char lookahead buffer for IN tokenization */
        storage INBUF[32];
        storage INBUF_VALID[8];
        view inbuf = INBUF;
        view inbuf_valid = INBUF_VALID[0];

        /* Status/flags */
        storage FLAGS[8];
        view ZF = FLAGS[0];


    memory:
        /* Bank 1: CODE + CONSTANTS (read-only) */
        range CODE_CONST[0x000000..0xFFFFFF]{
            cell = 8;
            endianess = little-endian;
            granularity = 8;
        }

        /* Bank 2: DATA (stack + globals + locals) */
        range DATA[0x0000..0xFFFF]{
            cell = 8;
            endianess = little-endian;
            granularity = 8;
```

```
        }

    /* Encodings (8-bit opcode + operands). */
    instructions:
        encode u8 field = immediate[8];
        encode u16 field = immediate[16];
        encode i16 field = immediate[16];
        encode i32 field = immediate[32];
        encode addr24 field = immediate[24];
        encode bool1 field = immediate[8];

        encode OPC field = cases {
            OP_PUSHI = 00000001,
            OP_PUSHB = 00000010,
            OP_PUSHC = 00000011,
            OP_LDG = 00000100,
            OP_STG = 00000101,
            OP_LDL = 00000110,
            OP_STL = 00000111,
            OP_DUP = 00001000,
            OP_POP = 00001001,
            OP_ADD = 00010000,
            OP_SUB = 00010001,
            OP_MUL = 00010010,
            OP_DIV = 00010011,
            OP_MOD = 00010100,
            OP_EQ = 00011000,
```

```
                OP_NE = 00011001,
                OP_LT = 00011010,
                OP_LE = 00011011,
                OP_GT = 00011100,
                OP_GE = 00011101,
                OP_AND = 00100000,
                OP_OR = 00100001,
                OP_JMP = 00110000,
                OP_JZ = 00110001,
                OP_JNZ = 00110010,
                OP_HALT = 00111111,
                OP_SETPORT = 01000000,
                OP_IN = 01000001,
                OP_OUT = 01000010
        };

        encode FMT_I32 sequence = {i32 as IMM};
        encode FMT_B1 sequence = {bool1 as IMM};
        encode FMT_U16 sequence = {u16 as IDX};
        encode FMT_I16 sequence = {i16 as OFF};
        encode FMT_J24 sequence = {addr24 as T};
        encode FMT_P16 sequence = {u16 as P};

        /* PUSHI: push i32 immediate onto stack */
        instruction PUSHI = { OPC.OP_PUSHI, sequence FMT_I32 } {
                if (sp == 0x10000) then sp = 0;
                /* Sign-extend 32-bit immediate to internal 64-bit value. */
```

```
                if ((IMM & 0x80000000) != 0) then
                        DATA[sp] = IMM | 0xFFFFFFFF00000000;
                else
                        DATA[sp] = IMM;
                sp = sp + 8;
                ip = ip + 5;
        };

        /* PUSHB: push bool immediate (0/1) */
        instruction PUSHB = { OPC.OP_PUSHB, sequence FMT_B1 } {
                if (sp == 0x10000) then sp = 0;
                DATA[sp] = IMM;
                sp = sp + 8;
                ip = ip + 2;
        };

        /* PUSHC: push constant by index */
        instruction PUSHC = { OPC.OP_PUSHC, sequence FMT_U16 } {
                if (sp == 0x10000) then sp = 0;
                DATA[sp] = CODE_CONST[IDX];
                sp = sp + 8;
                ip = ip + 3;
        };

        /* Globals */
        instruction LDG = { OPC.OP_LDG, sequence FMT_U16 } {
                if (sp == 0x10000) then sp = 0;
```

```
                DATA[sp] = DATA[0x2000 + IDX * 8];
                sp = sp + 8;
                ip = ip + 3;
        };
        instruction STG = { OPC.OP_STG, sequence FMT_U16 } {
                sp = sp - 8;
                DATA[0x2000 + IDX * 8] = DATA[sp];
                ip = ip + 3;
        };

        /* Locals relative to FP */
        instruction LDL = { OPC.OP_LDL, sequence FMT_I16 } {
                if (sp == 0x10000) then sp = 0;
                DATA[sp] = DATA[fp + OFF];
                sp = sp + 8;
                ip = ip + 3;
        };
        instruction STL = { OPC.OP_STL, sequence FMT_I16 } {
                sp = sp - 8;
                DATA[fp + OFF] = DATA[sp];
                ip = ip + 3;
        };

        /* Stack operations */
        instruction DUP = { OPC.OP_DUP } {
                DATA[sp] = DATA[sp - 8];
                sp = sp + 8;
```

```
                ip = ip + 1;
        };
        instruction POP = { OPC.OP_POP } {
                sp = sp - 8;
                ip = ip + 1;
        };

        /* Arithmetic */
        instruction ADD = { OPC.OP_ADD } {
                DATA[sp - 16] = DATA[sp - 16] + DATA[sp - 8];
                sp = sp - 8;
                ip = ip + 1;
        };
        instruction SUB = { OPC.OP_SUB } {
                DATA[sp - 16] = DATA[sp - 16] - DATA[sp - 8];
                sp = sp - 8;
                ip = ip + 1;
        };
        instruction MUL = { OPC.OP_MUL } {
                DATA[sp - 16] = DATA[sp - 16] * DATA[sp - 8];
                sp = sp - 8;
                ip = ip + 1;
        };
        instruction DIV = { OPC.OP_DIV } {
                DATA[sp - 16] = DATA[sp - 16] / DATA[sp - 8];
                sp = sp - 8;
                ip = ip + 1;
```

```
        };
        instruction MOD = { OPC.OP_MOD } {
                DATA[sp - 16] = DATA[sp - 16] % DATA[sp - 8];
                sp = sp - 8;
                ip = ip + 1;
        };

        /* Comparisons */
        instruction EQ = { OPC.OP_EQ } {
                DATA[sp - 16] = DATA[sp - 16] == DATA[sp - 8];
                ZF = DATA[sp - 16] == 0;
                sp = sp - 8;
                ip = ip + 1;
        };
        instruction NE = { OPC.OP_NE } {
                DATA[sp - 16] = DATA[sp - 16] != DATA[sp - 8];
                ZF = DATA[sp - 16] == 0;
                sp = sp - 8;
                ip = ip + 1;
        };
        instruction LT = { OPC.OP_LT } {
                let a = DATA[sp - 16];
                let b = DATA[sp - 8];
                let aNeg = (a & 0x8000000000000000) != 0;
                let bNeg = (b & 0x8000000000000000) != 0;

                if (aNeg && !bNeg) then
```

```
                DATA[sp - 16] = 1;
            else if (!aNeg && bNeg) then
                DATA[sp - 16] = 0;
            else
                DATA[sp - 16] = a < b;
            ZF = DATA[sp - 16] == 0;
            sp = sp - 8;
            ip = ip + 1;
        };
        instruction LE = { OPC.OP_LE } {
            let a = DATA[sp - 16];
            let b = DATA[sp - 8];
            let aNeg = (a & 0x8000000000000000) != 0;
            let bNeg = (b & 0x8000000000000000) != 0;

            if (aNeg && !bNeg) then
                DATA[sp - 16] = 1;
            else if (!aNeg && bNeg) then
                DATA[sp - 16] = 0;
            else
                DATA[sp - 16] = a <= b;
            ZF = DATA[sp - 16] == 0;
            sp = sp - 8;
            ip = ip + 1;
        };
        instruction GT = { OPC.OP_GT } {
            let a = DATA[sp - 16];
```

```
                let b = DATA[sp - 8];
                let aNeg = (a & 0x8000000000000000) != 0;
                let bNeg = (b & 0x8000000000000000) != 0;

                if (aNeg && !bNeg) then
                        DATA[sp - 16] = 0;
                else if (!aNeg && bNeg) then
                        DATA[sp - 16] = 1;
                else
                        DATA[sp - 16] = a > b;
                ZF = DATA[sp - 16] == 0;
                sp = sp - 8;
                ip = ip + 1;
        };
        instruction GE = { OPC.OP_GE } {
                let a = DATA[sp - 16];
                let b = DATA[sp - 8];
                let aNeg = (a & 0x8000000000000000) != 0;
                let bNeg = (b & 0x8000000000000000) != 0;

                if (aNeg && !bNeg) then
                        DATA[sp - 16] = 0;
                else if (!aNeg && bNeg) then
                        DATA[sp - 16] = 1;
                else
                        DATA[sp - 16] = a >= b;
                ZF = DATA[sp - 16] == 0;
```

```
                sp = sp - 8;
                ip = ip + 1;
        };

        /* Boolean */
        instruction AND = { OPC.OP_AND } {
                DATA[sp - 16] = (DATA[sp - 16] != 0) && (DATA[sp - 8] != 0);
                ZF = DATA[sp - 16] == 0;
                sp = sp - 8;
                ip = ip + 1;
        };
        instruction OR = { OPC.OP_OR } {
                DATA[sp - 16] = (DATA[sp - 16] != 0) || (DATA[sp - 8] != 0);
                ZF = DATA[sp - 16] == 0;
                sp = sp - 8;
                ip = ip + 1;
        };

        /* Jumps */
        instruction JMP = { OPC.OP_JMP, sequence FMT_J24 } {
                ip = T;
        };
        instruction JZ = { OPC.OP_JZ, sequence FMT_J24 } {
                sp = sp - 8;
                if (DATA[sp] == 0) then ip = T;
                else ip = ip + 4;
        };
```

```
instruction JNZ = { OPC.OP_JNZ, sequence FMT_J24 } {
        sp = sp - 8;
        if (DATA[sp] != 0) then ip = T;
        else ip = ip + 4;
};

/* HALT */
instruction HALT = { OPC.OP_HALT } {
        break;
};

/* IO */
instruction SETPORT = { OPC.OP_SETPORT, sequence FMT_P16 } {
        ioport = P;
        ip = ip + 3;
};
instruction IN = { OPC.OP_IN } {
        if (sp == 0x10000) then sp = 0;

        /*
         * Read one token:
         * - decimal integer (with optional unary '-')
         * - operator symbol '+', '-', '*', '/' as ASCII code
         * - any other single non-space symbol as ASCII code
         */
        let c = 0;
        if (inbuf_valid != 0) then {
```

```
                c = inbuf;
                inbuf_valid = 0;
            }
            else c = stdin;

            while (c == 32 || c == 10 || c == 13 || c == 9) do
                c = stdin;

            if (c == 43 || c == 42 || c == 47) then
                DATA[sp] = c;
            else if (c == 45) then {
                let c2 = stdin;
                if (c2 >= 48 && c2 <= 57) then {
                    let n = c2 - 48;
                    let d = stdin;
                    while (d >= 48 && d <= 57) do {
                        n = n * 10 + (d - 48);
                        d = stdin;
                    }
                    if (!(d == 32 || d == 10 || d == 13 || d == 9)) then {
                        inbuf = d;
                        inbuf_valid = 1;
                    }
                    DATA[sp] = (0 - n) & 0xFFFFFFFFFFFFFFFF;
                }
                else {
                    DATA[sp] = 45;
```

```
                    if (!(c2 == 32 || c2 == 10 || c2 == 13 || c2 == 9)) then {
                        inbuf = c2;
                        inbuf_valid = 1;
                    }
                }
            }
            else if (c >= 48 && c <= 57) then {
                let n = 0;
                while (c >= 48 && c <= 57) do {
                    n = n * 10 + (c - 48);
                    c = stdin;
                }
                if (!(c == 32 || c == 10 || c == 13 || c == 9)) then {
                    inbuf = c;
                    inbuf_valid = 1;
                }
                DATA[sp] = n;
            }
            else
                DATA[sp] = c;

            sp = sp + 8;
            ip = ip + 1;
        };
        /*
         * OUT prints stack top as decimal text.
         * Example: 65 -> bytes '6' and '5' (0x36 0x35).
```

```
*/
instruction OUT = { OPC.OP_OUT } {
    sp = sp - 8;
    let n = DATA[sp];

    if (n == 0) then {
        stdout = 48; /* '0' */
    }
    else {
        /* DATA is effectively 64-bit. Interpret sign via high bit. */
        if ((n & 0x8000000000000000) != 0) then {
            stdout = 45; /* '-' */
            /* two's complement absolute value */
            n = ((~n) + 1) & 0xFFFFFFFFFFFFFFFF;
        }

        /* Store decimal digits in reverse order, then print back. */
        let count = 0;
        while (n > 0) do {
            DATA[0x4000 + count * 8] = n % 10;
            n = n / 10;
            count = count + 1;
        }

        while (count > 0) do {
            count = count - 1;
            let d = DATA[0x4000 + count * 8];
```

```
                                        stdout = d + 48;
                                }
                        }

                        ip = ip + 1;
                };


        /* (iii) MNEMONICS */
        mnemonics:
                format NONE is "";
                format IMM32 is "{1}";
                format IMM1 is "{1}";
                format IDX16 is "{1}";
                format OFF16 is "{1}";
                format T24 is "{1}";
                format P16 is "{1}";

                mnemonic pushi for PUSHI (IMM) IMM32;
                mnemonic pushb for PUSHB (IMM) IMM1;
                mnemonic pushc for PUSHC (IDX) IDX16;

                mnemonic ldg for LDG (IDX) IDX16;
                mnemonic stg for STG (IDX) IDX16;

                mnemonic ldl for LDL (OFF) OFF16;
                mnemonic stl for STL (OFF) OFF16;
```

```
mnemonic dup for DUP () NONE;
mnemonic pop for POP () NONE;

mnemonic add for ADD () NONE;
mnemonic sub for SUB () NONE;
mnemonic mul for MUL () NONE;
mnemonic div for DIV () NONE;
mnemonic mod for MOD () NONE;

mnemonic eq for EQ () NONE;
mnemonic ne for NE () NONE;
mnemonic lt for LT () NONE;
mnemonic le for LE () NONE;
mnemonic gt for GT () NONE;
mnemonic ge for GE () NONE;

mnemonic and_ for AND () NONE;
mnemonic or_ for OR () NONE;

mnemonic jmp for JMP (T) T24;
mnemonic jz for JZ (T) T24;
mnemonic jnz for JNZ (T) T24;
mnemonic halt for HALT () NONE;

mnemonic setport for SETPORT (P) P16;
mnemonic in for IN () NONE;
```

```
                    mnemonic out for OUT () NONE;
}
```

Итоговая структура данных, которая получается при обработке набора подпрограмм – SubprogramImage, содержащая набор инструкций и элементов данных, каждая инструкция Instruction при этом содержит мнемонику и операнды. В качестве основной функции используется SubprogramImage* toAsmModule(const SubprogramInfo* info), которая по описанию подпрограммы собирает элемент образа программы. В свою очередь функция bool generateProgramAsm(const SubprogramCollection* subprograms, FILE* out, char** error_message) генерирует по набору подпрограмм итоговый asm файл. Остальные функции являются вспомогательными. Код для представления структуры данных и объявленные методы находятся в файле to_asm_module.h и представлен ниже:

```
#include <stdbool.h>

#include "cfg_builder_module.h"

typedef enum {
    DATA_ITEM_LITERAL,
    DATA_ITEM_TYPE_SIZE
} DataItemKind;

typedef struct {
    DataItemKind kind;
    union {
        char* literal_name;
        int size_bytes;
    } value;
} DataItem;

typedef struct {
    char* mnemonic;
    char** operands;
    int operand_count;
```

```
} Instruction;

typedef struct {
    DataItem* data_items;
    int data_item_count;
    Instruction* instructions;
    int instruction_count;
} SubprogramImage;

SubprogramImage* toAsmModule(const SubprogramInfo* info);
bool generateProgramAsm(const SubprogramCollection* subprograms,
FILE* out, char** error_message);
void freeSubprogramImage(SubprogramImage* image);
void printSubprogramImage(const SubprogramImage* image, const
char* entry_label, FILE* out);
void printSubprogramImageConsole(const SubprogramImage* image,
const char* entry_label);

#endif
```

2 Реализация модуля, генерирующего линейный код по набору подпрограмм.

Для генерации линейного кода сначала производится поиск по набору подпрограмм с целью найти main метод, затем проверяется наличие тела у метода, а также проверяется граф вызовов. После чего строится asm листинг для программы (точкой входа считается main метод).

Код созданной утилиты находится в файле to_asm_module.c, ниже приведены несколько основополагающих методов созданного класса:

```
bool generateProgramAsm(const SubprogramCollection* subprograms, FILE* out, char**
error_message)
{
    if (error_message) {
        *error_message = NULL;
    }
```

```
    if (!out) {
        out = stdout;
    }

    if (!subprograms) {
        if (error_message) {
            *error_message = strdup("Subprogram collection is null.");
        }
        return false;
    }

    const SubprogramInfo* main_method = find_main_method(subprograms);
    if (!main_method) {
        fprintf(out, "[section CODE_CONST]\n");
        fprintf(out, "halt\n");
        return true;
    }

    ReturnSiteList return_sites;
    return_site_list_init(&return_sites);

    SubprogramImage** images = calloc(subprograms->count, sizeof(SubprogramImage*));
    if (!images) {
        if (error_message) {
            *error_message = strdup("Out of memory while preparing ASM images.");
        }
        return_site_list_free(&return_sites);
        return false;
    }

    for (int i = 0; i < subprograms->count; i++) {
        const SubprogramInfo* info = &subprograms->items[i];
        if (!info || !info->name || is_method_builtin_declaration(info)) {
            continue;
        }

        bool is_main = (info == main_method);
        char* local_error = NULL;
        images[i] = toAsmModuleInternal(info, subprograms, &return_sites, is_main, false,
&local_error);
        if (!images[i]) {
            if (error_message) {
                if (local_error) {
                    *error_message = local_error;
```

```
            } else {
                *error_message = strdup("Failed to generate ASM image.");
            }
        } else {
            free(local_error);
        }

        for (int j = 0; j < subprograms->count; j++) {
            freeSubprogramImage(images[j]);
        }
        free(images);
        return_site_list_free(&return_sites);
        return false;
    }
  }

  fprintf(out, "[section CODE_CONST]\n");
  fprintf(out, "\n");

  for (int pass = 0; pass < 2; pass++) {
    for (int i = 0; i < subprograms->count; i++) {
        const SubprogramInfo* info = &subprograms->items[i];
        if (!info || !info->name || is_method_builtin_declaration(info)) {
            continue;
        }

        bool is_main = (info == main_method);
        if ((pass == 0 && !is_main) || (pass == 1 && is_main)) {
            continue;
        }

        if (!images[i]) {
            continue;
        }

        printSubprogramImage(images[i], info->name, out);
        fprintf(out, "\n");
    }
  }

  if (return_sites.count > 0) {
    fprintf(out, "%s:\n", RUNTIME_DISPATCH_LABEL);
    for (int i = 0; i < return_sites.count; i++) {
        const ReturnSite* site = &return_sites.items[i];
```

```
            fprintf(out, "    dup\n");
            fprintf(out, "    pushi %d\n", site->id);
            fprintf(out, "    eq\n");
            fprintf(out, "    jnz M_sys_ret_case_%d\n", site->id);
        }
        fprintf(out, "    pop\n");
        fprintf(out, "    halt\n");

        for (int i = 0; i < return_sites.count; i++) {
            const ReturnSite* site = &return_sites.items[i];
            fprintf(out, "M_sys_ret_case_%d:\n", site->id);
            fprintf(out, "    pop\n");
            fprintf(out, "    jmp %s\n", site->continue_label ? site->continue_label : "");
        }
        fprintf(out, "\n");
    }

    for (int i = 0; i < subprograms->count; i++) {
        freeSubprogramImage(images[i]);
    }
    free(images);
    return_site_list_free(&return_sites);

    return true;
}
```

Для запуска ассемблированного кода на виртуальной машине необходимо выполнить ряд команд (часть из них необходима, другая использовалась для отладки). Для ассемблирования asm в binary code используется команда .\Portable.RemoteTasks.Manager.exe -ul 507031 -up 9f8bfd81-95f3-4a10-9ff8-3097ed5cd581 -w -s Assemble definitionFile .\target-definitions.pdsl archName MyVM asmListing .\listing.asm. Для получения информации по выполненной таске используется команда .\Portable.RemoteTasks.Manager.exe -ul 507031 -up 9f8bfd81-95f3-4a10-9ff8-3097ed5cd581 -g ff1fe17c-a148-4b0d-9ba8-38c7cd196d86. Для получения бинарника и записи в файл result.ptptb используется команда .\Portable.RemoteTasks.Manager.exe -ul 507031 -up 9f8bfd81-95f3-4a10-9ff8-3097ed5cd581 -g 20df03c2-20d5-4a0b-9b6e-b34763cb7bef -r out.ptptb -o result.ptptb. Для дизассемблирования используется команда .\Portable.RemoteTasks.Manager.exe -ul 507031 -up 9f8bfd81-95f3-4a10-9ff8-3097ed5cd581 -w -s Disassemble definitionFile .\target-definitions.pdsl

archName MyVM in.ptptb .\result.ptptb. Просмотр результата дизассемблирования производится командой .\Portable.RemoteTasks.Manager.exe -ul 507031 -up 9f8bfd81-95f3-4a10-9ff8-3097ed5cd581 -g a7c12c48-bdee-442e-9cac-0e0dad6fdcad -r disasmListing.txt -o disassembled.txt. Выполняем бинарник командой .\Portable.RemoteTasks.Manager.exe -ul 507031 -up 9f8bfd81-95f3-4a10-9ff8-3097ed5cd581 -w -s ExecuteBinaryWithInteractiveInput definitionFile .\target-definitions.pdsl archName MyVM binaryFileToRun .\result.ptptb codeRamBankName CODE_CONST ipRegStorageName IP finishMnemonicName halt stdinRegStName STDIN stdoutRegStName STDOUT. Результаты получаются командой .\Portable.RemoteTasks.Manager.exe -ul 507031 -up 9f8bfd81-95f3-4a10-9ff8-3097ed5cd581 -g 3e74f3ca-bf3e-4fa5-a2e0-01443e4a7d80 -r stdout.txt -o out.txt. Trace производится командой .\Portable.RemoteTasks.Manager.exe -ul 507031 -up 9f8bfd81-95f3-4a10-9ff8-3097ed5cd581 -g 3e74f3ca-bf3e-4fa5-a2e0-01443e4a7d80 -r trace.txt -o trace.txt.

3 Тестирование модуля на работоспособность

Для тестирования модуля на работоспособность разработан сложный пример, содержащий в себе как программу калькулятор, так и программу для подсчёта n-го числа Фибоначчи.

Входной файл (calc.txt):

```
method calc(x : int, op : char, y : int)
var result : int;
begin
   if op == '+' then result := x + y;
   if op == '-' then result := x - y;
   if op == '*' then result := x * y;
   if op == '/' then result := x / y;
   write(result);
end;

method fib(n : int) : int
var result : int;
begin
   if n <= 2 then
      result := 1;
   else
```

```
        result := fib(n - 2) + fib(n - 1);

    result;
end;

method main()
var x, y : int;
op : char;
begin
    x := read();
    op := read();
    if op == 'f' then
        write(fib(x));
    else begin
        y := read();
        calc(x, op, y);
    end;
end;


method read();
method write(num : int);
```

По входному файлу производим ручное тестирование программы.

Вывод:

В ходе практической работы разработана программа для построения линейного кода программы на основе набора описания подпрограмм. В результате полученный asm листинг можно использовать для ассемблирования и выполнения на кастомной виртуальной машине при помощи передачи целевой архитектуры (.pdsl файла), наименования банка памяти и прочих настроек.